

Introduction à l'algorithmique et à la programmation

Notions de programmation en C

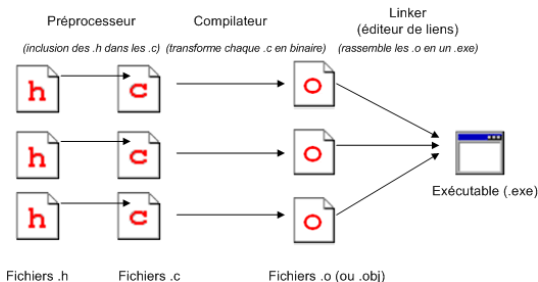
Département informatique - IUT de Lannion

Rappel

Pour obtenir un programme exécutable :

- Compiler le programme source
- Faire une édition de liens du résultat de la compilation

Programme source → Compilation → Edition de liens → Programme exécutable



Plan

- ① La compilation
- ② Le code source
- ③ Les types
- ④ Entrée-sortie

La compilation

- Compilation : une fois le fichier source écrit `premier.c` dans un éditeur de texte
- Compilation : par un compilateur `cc` (autre compilateur en C `gcc`)
- La commande dans le terminal : `cc premier.c` effectue :
 - un preprocessing : un fichier `premier.i` est créé
 - une compilation : un fichier `premier.s` est créé
 - un assemblage : un fichier `premier.o` est créé
 - le lien : le fichier `a.out` est créé
- Seul le dernier fichier reste présent
- Exécution du programme : `./a.out`

La compilation : les options

- La commande : `man cc` précise ce que fait `cc` et donne les options :
 - `-o` nomme le fichier de sortie
 - `-Wall` active tous (*all*) les messages d'avertissement (*Warning*)
- La bonne commande :
`cc premier.c -o premier.o -Wall`

Plan

- ① La compilation
- ② Le code source
- ③ Les types
- ④ Entrée-sortie

Le code source

- Le nom du fichier du code source :
 - ne pas mettre de majuscule à l'initiale,
 - ne pas mettre d'accent,
 - se finit par `.c`
- Dans le fichier `premier.c` :

```
// inclusion des bibliothèques
#include <stdio.h>
#include <stdlib.h>
// Un essai de programme
int main()
{
    // définition des variables
    // les instructions
    return EXIT_SUCCESS;
}
```

Le code source : Quelques règles

- Règles syntaxiques
 - point virgule après chaque instruction
 - accolades (ouvrante et fermante) en début et fin de chaque bloc d'instructions (éventuellement en fin de ligne pour { })
- La syntaxe d'un programme est définie par une grammaire
- Des mots réservés : `main`, `char`, `double`, `float`, `const`, `if`, `else`, etc.
- Des symboles spéciaux :
– `*` `+` `/` `!` `;` `>` `<` `=` `||` `&&`
- Commentaires :
 - `//` : commente la ligne
 - `/* commentaire */` : commente plusieurs lignes
- Toute variable doit être déclarée

Le code source : Les variables

Déclaration

- Rappel : sert à mettre l'étiquette (nom de la boîte)
- Syntaxe : `<type> <nom_variable>;`
- `nom_variable` :
 - ne commence pas par un chiffre
 - longueur maximale : 31 caractères
 - conventions
 - ne commence pas par une majuscule
 - ne pas mettre d'accent
 - mots composés : `nom_variable` ou `nomVariable`

Affectation

Pour les entiers, réels, caractères et booléens :

`<variable> = <valeur>;` de la droite vers la gauche

Rappel en pseudo-code `<variable> := <valeur>;`

Plan

- ① La compilation
- ② Le code source
- ③ **Les types**
- ④ Entrée-sortie

Le type entier

- Pour manipuler les entiers, C propose 6 types.
- Les opérations sur les entiers sont $+$, $-$, $*$, la division entière $/$, $\%$ reste de la division. Exemple :
 - $9/2$ donne 4
 - $9\%2$ donne 1

type	signification	codage (octets)	valeur
short int	Entier court	2	-32 768 à 32 767
unsigned short int	Entier court non signé	2	0 à 65 535
int	Entier	2 (pro. 16 bits) 4 (pro. 32 bits)	-32 768 à 32 767 -2 147 483 648 à 2 147 483 647
unsigned int	Entier non signé	2 (pro. 16 bits) 4 (pro. 32 bits)	0 à 65 535 0 à 4 294 967 295
long int	Entier long	4	-2 147 483 648 à 2 147 483 647
unsigned long int	Entier long non signé	4	0 à 4 294 967 295

Le type caractère

- N'existe pas de manière indépendante en C
- Représenté par des `char` correspondant au codage ASCII des caractères
- la lettre 'a' sera représentée par l'entier 97 ! Une erreur flagrante de typage (`4 * 'a'`) ne sera pas détectée par le compilateur

type	signification	codage (octets)	valeur
char	Caractère	1 (8 bits)	-128 à 127
unsigned char	Caractère non signé	1	0 à 255

Recommandation : bien distinguer

```
int n;  
char c;
```

Le type réel

- Pour manipuler les réels, C propose 3 types (d'autres existent) présentés dans la table ci-dessous
- Les opérateurs sur les réels : $+$, $-$, $*$, $/$
Exemple : $9.0/2.0$ donne 4.5

type	signification	codage	valeur	précision
float	réel simple	octets 4	$3.4 * 10^{-38}$ à $3.4 * 10^{38}$	7 chiffres
double	réel double	8	$1.7 * 10^{-308}$ à $1.7 * 10^{308}$	15 chiffres
long double	réel double long	10	$3.4 * 10^{-4932}$ à $3.4 * 10^{4932}$	17 chiffres

Le type chaîne et booléen

Chaîne

- N'existe pas de manière indépendante en C
- Il faut définir un tableau (nous verrons ce que c'est) de caractères
- `char nom[50];` //chaîne de 50 caractères

Booléen

- N'existe pas de manière indépendante en C : il faut ajouter une librairie `#include <stdbool.h>`
- `bool unBooleen;`
- `unBooleen = true;` équivaut à 1
- `unBooleen = false;` équivaut à 0
- Les opérateurs :

&& ou => || non => ! xor =

Déclarations : Exemples

- des variables dans le main

```
int note, longueur;  
char prenom[10];
```

- des constantes avant le main

```
const char LETTRE_MAJ = 'B';  
const int N = 3;  
const float PI = 3.14;  
const char MON_NOM [6] = "Martin";
```

Affectations : Exemples

```
int main()  
{  
    int note, longueur;  
    char letter;  
    bool test;  
    note=20;  
    longueur = 8;  
    lettre = 'B';  
    test=true;  
    return EXIT_SUCCESS;  
}
```

Bien utiliser des chaînes

- Les chaînes de caractères sont des constantes particulières car elles se manipulent avec des primitives (des instructions de la librairie `string.h`)

Si `ch1` et `ch2` sont deux chaînes de caractères

- **Concaténation** : la concaténation de `ch2` et `ch1` se fait avec l'instruction `strcat(ch2, ch1)`
 - En langage C : `strcat(ch2, ch1);` attention à la taille de `ch2`
 - En pseudo-code : `ch2 := ch2 + ch1;`
- **Copie** : la copie de `ch2` et `ch1` se fait avec l'instruction `strcpy(ch2, ch1)`
 - En langage C : `strcpy(ch, "Bonjour");`
 - En pseudo-code : `ch := "Bonjour";`
- **Comparasion** : la comparaison de `ch1` et `ch2` se fait avec l'instruction `n = strcmp(ch1, ch2)`

Plan

- ① La compilation
- ② Le code source
- ③ Les types
- ④ **Entrée-sortie**

Instructions d'entrée-sortie

- Pour afficher une chaîne de caractères :

```
printf("Bonjour");
```

- Si on affiche le contenu de variables, on doit les insérer dans une chaîne de caractères (écriture formatée) :

```
int a, b;
```

```
a=9;
```

```
b=8;
```

```
printf("%d %d", a, b);
```

```
printf("la valeur de a : %d et celle de b : %d", a, b);
```

- Pour insérer les variables dans une chaîne de caractères :

%d pour int, %s pour les chaînes, %f pour les float, %c pour les caractères

- caractère particulier : '\n' pour <RETURN>

Instructions d'entrée-sortie

- Pour lire au clavier une variable : utiliser l'instruction `scanf` et indiquer le format
`%d` pour int, `%s` pour les chaînes, `%f` pour les float, `%c` pour les caractères
- et donner l'adresse de la variable pour la modifier
- L'adresse s'obtient avec le `&` (sauf pour les chaînes de caractères)
- Exemple

```
int a;  
// &a est l'adresse de la variable a  
scanf("%d",&a);  
// la machine attend que l'utilisateur  
// frappe un entier au clavier ...
```

Instructions d'entrée-sortie

- Dès que le programme rencontre un `scanf` il attend la frappe d'une valeur au clavier.
- `scanf("%d %f %s",&a, &b, nom);` permet de lire trois valeurs au clavier.
- Pour une chaîne on ne met pas le `'&'`.

“Un programme est un sort jeté sur un ordinateur, qui transforme tout texte saisi au clavier en message d'erreur.” - Anonyme

Un exemple

```
#include <stdio.h>
#include <stdlib.h>
    // Un essai de programme
int main()
{
    // les variables du programme
    char n[50], pr[50]; // au plus 49 caractères
    printf("Votre nom : ");
    scanf("%s",n); // l'utilisateur donne un nom
                   // mis dans la variable n
    printf("Votre prénom : ");
    scanf("%s",pr); // saisie du prénom
                   // mis dans la variable pr
    printf("bonjour %s %s", pr, n);
    return EXIT_SUCCESS;
}
```